

Genetic Algorithms

Nirjhor

Department of Computer Science, BRACU

Why Genetic Algorithms?

Optimization Problem

$$\max_{x \in \mathcal{X}} f(x)$$

- $\mathcal{X}$ may be:
 - discrete
 - non-convex
 - non-differentiable
 - combinatorial
- Gradient-based methods fail when:
 - $\nabla f(x)$ does not exist
 - objective is black-box

Core Idea

Optimize by evolving a **population** of solutions using selection, crossover, and mutation.

Biological Inspiration (Formalized)

- Individual $\leftrightarrow$ candidate solution
- Gene $\leftrightarrow$ decision variable
- Chromosome $\leftrightarrow$ encoded vector
- Fitness $\leftrightarrow$ objective function

Population at generation t : $P_t = \{x_1^{(t)}, \dots, x_N^{(t)}\}$

Biological Evolution Model

- Population of individuals
- Genetic variation (mutation + recombination)
- Differential survival (natural selection)
- Reproduction proportional to fitness

Representation (Encoding)

Let $x \in \mathcal{X}$ be a solution.

Common Encodings

- Binary: $x \in \{0, 1\}^L$
- Real-valued: $x \in \mathbb{R}^d$
- Permutation: $x \in S_n$

Key Principle

Encoding determines:

- feasible genetic operators
- search geometry

Example 1: Feature Selection (Binary Encoding)

Problem

Given L features, select the subset that maximizes accuracy.

$$\max_{x \in \{0,1\}^L} f(x)$$

Encoding

$$x = (x_1, x_2, \dots, x_L), \quad x_i \in \{0, 1\}$$

- $x_i = 1 \rightarrow$ select feature i
- $x_i = 0 \rightarrow$ discard feature i

Genetic Operators

- Mutation: flip bit
- Crossover: swap bit segments

Example 2: Hyperparameter Optimization (Real Encoding)

Problem

Tune model parameters to minimize validation loss.

$$\min_{x \in \mathbb{R}^d} f(x)$$

Encoding

$$x = (\text{learning rate}, \text{dropout}, \text{depth})$$

Genetic Operators

- Mutation: $x_i \leftarrow x_i + \epsilon$
- Crossover: $x_{child} = \alpha x_1 + (1 - \alpha)x_2$

Example 3: Traveling Salesman Problem (Permutation Encoding)

Problem

Find shortest tour visiting each city exactly once.

$$\min_{x \in S_n} f(x)$$

Encoding

$$x = (3, 5, 1, 4, 2)$$

- Each city appears exactly once

Genetic Operators

- Mutation: swap two cities
- Crossover: Order Crossover (OX), PMX

Fitness Function

Define fitness:

$$F(x) : \mathcal{X} \rightarrow \mathbb{R}^+$$

For minimization:

$$F(x) = \frac{1}{1 + f(x)}$$

- Fitness is an **evaluation**, not probability
- Fitness may be noisy or stochastic
- Fitness landscape may be deceptive

Selection Mechanisms

Goal:

$$P(\text{select } x_i) \uparrow \text{ as } F(x_i) \uparrow$$

Roulette-Wheel Selection

$$P(x_i) = \frac{F(x_i)}{\sum_{j=1}^N F(x_j)}$$

Tournament Selection

Randomly sample k individuals, select the best.

$$P(\text{best}) = 1 - (1 - p)^k$$

Selection Examples: Roulette vs Tournament

Population Fitness

Individual	Fitness
A	10
B	30
C	60

Roulette-Wheel Selection

$$P(A) = 0.10, \quad P(B) = 0.30, \quad P(C) = 0.60$$

Tournament Selection ($k = 2$)

- Randomly pick 2 individuals
- Select the one with higher fitness
- Higher $k \Rightarrow$ stronger selection pressure

Selection Example: Population of 4 Individuals

Individual	Fitness
A	5
B	15
C	30
D	50

Roulette Selection

$$P(A) = 0.05, \quad P(B) = 0.15, \quad P(C) = 0.30, \quad P(D) = 0.50$$

Random draw 0.72 $\rightarrow$ select D

Tournament Selection ($k = 2$)

Randomly pick (B,C) $\rightarrow$ C wins

Crossover (Recombination)

Let parents $x^{(1)}, x^{(2)}$.

Binary Single-Point Crossover

$$x^{(c)} = (x_1^{(1)}, \dots, x_k^{(1)}, x_{k+1}^{(2)}, \dots)$$

Real-Valued (Arithmetic) Crossover

$$x^{(c)} = \alpha x^{(1)} + (1 - \alpha)x^{(2)}, \quad \alpha \in [0, 1]$$

Binary Double-Point Crossover

Let parents:

$$x^{(1)} = (10110011)$$

$$x^{(2)} = (01001100)$$

Choose crossover points $k_1 = 3$, $k_2 = 6$

Segments

$$x^{(1)} = (101 \mid 100 \mid 11)$$

$$x^{(2)} = (010 \mid 011 \mid 00)$$

Child

$$x^{(c)} = (101 \mid 011 \mid 11)$$

Mutation

Mutation introduces random variation.

Binary Mutation

$$x_i \leftarrow 1 - x_i \quad \text{with prob } p_m$$

Gaussian Mutation

$$x_i \leftarrow x_i + \epsilon, \quad \epsilon \sim \mathcal{N}(0, \sigma^2)$$

- Prevents premature convergence
- Enables exploration

Canonical Genetic Algorithm

Algorithm 1 Genetic Algorithm

[1] Initialize population P_0

for $t = 0$ to T **do**

 Evaluate fitness $F(x)$ for all $x \in P_t$

 Select parents from P_t

 Apply crossover

 Apply mutation

 Form new population P_{t+1}

end for

return best individual found

Scenario: Drone Design Optimization

Goal

Maximize flight distance of delivery drone.

Encoding (3 genes)

- Gene 1: High-power motor
- Gene 2: Lightweight frame
- Gene 3: Advanced battery

Example Chromosome

101

Motor = Yes, Frame = No, Battery = Yes

Step 1: Initial Population

Chromosome	Flight Distance (km)
100	30
011	40
101	50
010	20

Total Fitness = 140

Step 2: Selection

$$P(x_i) = \frac{F(x_i)}{\sum F(x)}$$

Chromosome	Probability
100	0.21
011	0.29
101	0.36
010	0.14

Example selection:

Parents: 101 and 011

Crossover: Combining Good Traits

Selected Parents

101 (50 km)

011 (40 km)

Single-Point Crossover (cut after first gene)

1|01

0|11

Child

1|11 = 111

New Drone Design:

- Motor: Yes

Mutation: Introducing Variation

Child Before Mutation

111

Mutation Probability = 0.1

Suppose gene 2 flips:

111 $\rightarrow$ 101

Effect of Mutation

- Introduces diversity
- Prevents premature convergence
- Allows exploration of new designs

Key Insight

Genetic Algorithms are not random search. They are structured, probabilistic, population-based optimization methods grounded in evolutionary theory.